



Midterm Review

CS 400 – Programming III

Matt Schwennesen

July 09, 2026

Outline

Midterm Logistics

Midterm Review

Practice Exam



Midterm Logistics

Midterm Logistics

- ◆ Midterm released Thursday, July 09, 2026 at 12:00:00 AM Central Time

Midterm Logistics

- ◆ Midterm released Thursday, July 09, 2026 at 12:00:00 AM Central Time
- ◆ Midterm due Friday, July 10, 2026 at 11:59:59 PM Central Time

Midterm Logistics

- ◆ Midterm released Thursday, July 09, 2026 at 12:00:00 AM Central Time
- ◆ Midterm due Friday, July 10, 2026 at 11:59:59 PM Central Time
- ◆ Two hour time limit.

Midterm Logistics

- ◆ Midterm released Thursday, July 09, 2026 at 12:00:00 AM Central Time
- ◆ Midterm due Friday, July 10, 2026 at 11:59:59 PM Central Time
- ◆ Two hour time limit.
- ◆ 30 questions, no written code questions.

Midterm Logistics

- ◆ Midterm released Thursday, July 09, 2026 at 12:00:00 AM Central Time
- ◆ Midterm due Friday, July 10, 2026 at 11:59:59 PM Central Time
- ◆ Two hour time limit.
- ◆ 30 questions, no written code questions.
- ◆ Scratch paper allowed and recommended.

Midterm Logistics

- ◆ Midterm released Thursday, July 09, 2026 at 12:00:00 AM Central Time
- ◆ Midterm due Friday, July 10, 2026 at 11:59:59 PM Central Time
- ◆ Two hour time limit.
- ◆ 30 questions, no written code questions.
- ◆ Scratch paper allowed and recommended.
- ◆ Honorlock required.
 - ◇ Google chrome based browser with Honorlock extension

Midterm Topics

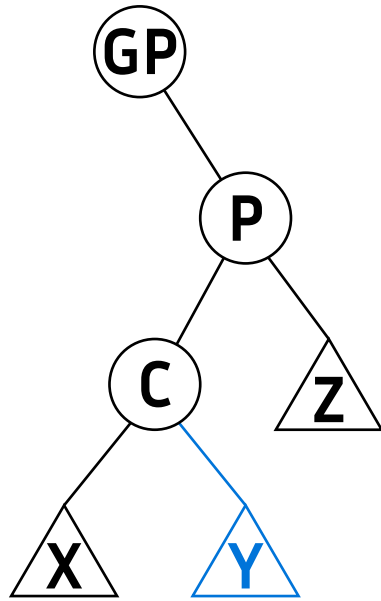
- ✦ Binary Search Tree
 - Rotations
- ✦ Red-Black Trees
 - ◊ Insertion
 - ◊ Deletion
 - ◊ Properties
- ✦ AVL trees
- ✦ B-Trees
- ✦ Graphs
- ✦ CSV files
- ✦ Make
- ✦ Testing & JUnit
- ✦ Bash Commands
- ✦ Version Control
- ✦ Anonymous Classes
- ✦ Lambda Expressions



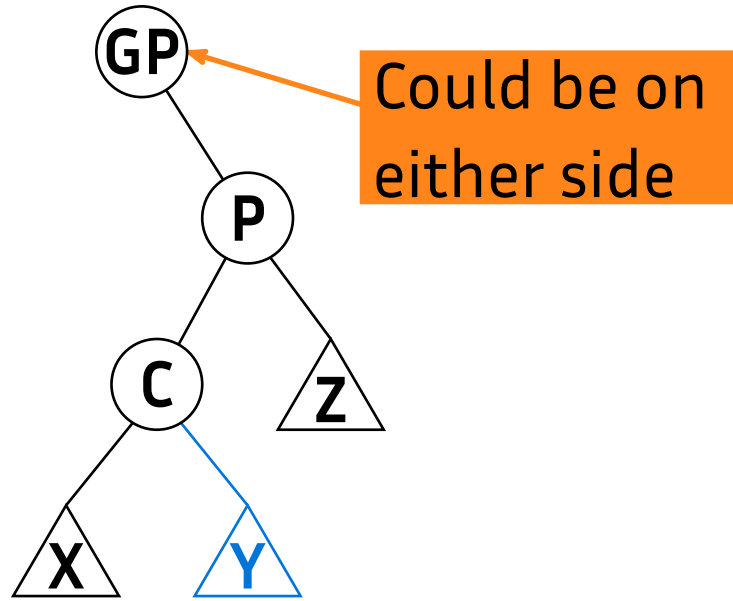
Midterm Review

BST Right Rotation Pattern

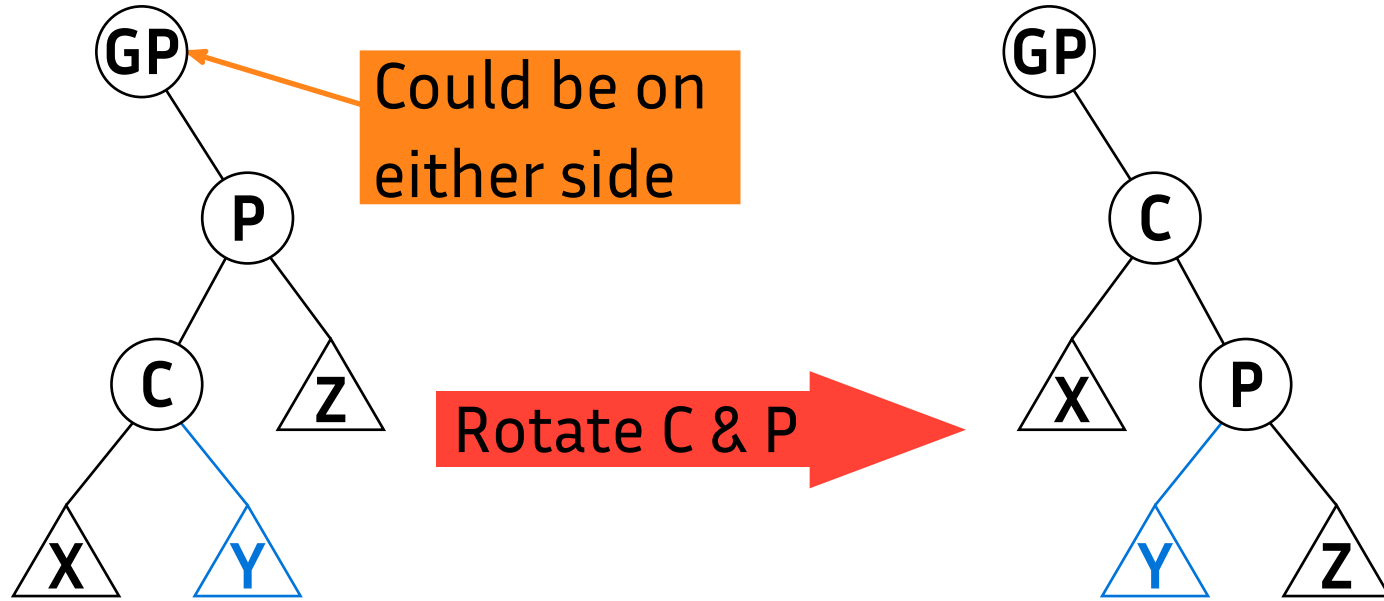
BST Right Rotation Pattern



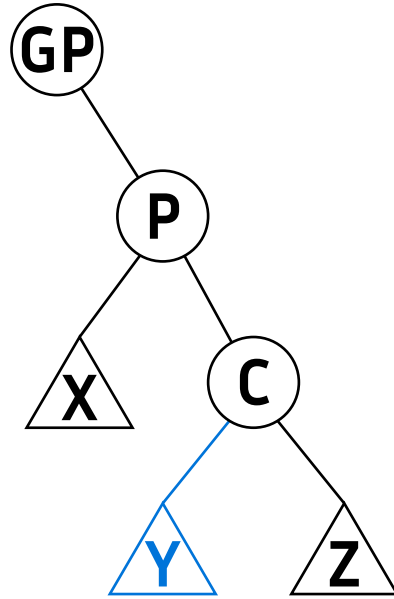
BST Right Rotation Pattern



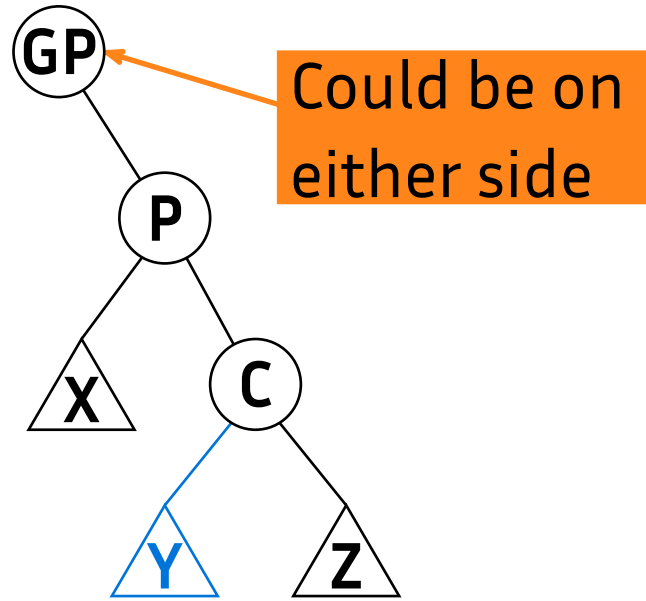
BST Right Rotation Pattern



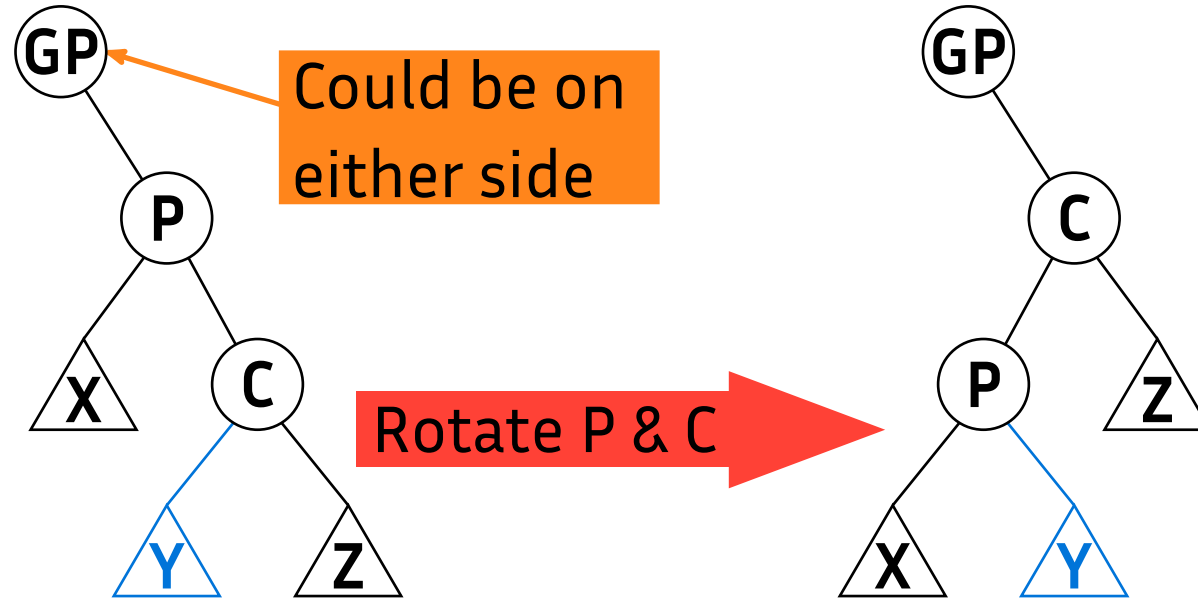
BST Left Rotation Pattern



BST Left Rotation Pattern



BST Left Rotation Pattern



Red Black Tree Properties

Definition (Red Black Tree) A binary search tree with four extra properties:

- 1.
- 2.
- 3.
- 4.

Red Black Tree Properties

Definition (Red Black Tree) A binary search tree with four extra properties:

1. Color Property
- 2.
- 3.
- 4.

Red Black Tree Properties

Definition (Red Black Tree) A binary search tree with four extra properties:

1. Color Property
2. Red-Child Property
- 3.
- 4.

Red Black Tree Properties

Definition (Red Black Tree) A binary search tree with four extra properties:

1. Color Property
2. Red-Child Property
3. Root Property
- 4.

Red Black Tree Properties

Definition (Red Black Tree) A binary search tree with four extra properties:

1. Color Property
2. Red-Child Property
3. Root Property
4. Black-Height Property

Red Black Tree Insertions

1. Start with a **valid** red black tree.

Red Black Tree Insertions

1. Start with a **valid** red black tree.
2. Perform **naive BST insertion** algorithm, ignoring colors.

Red Black Tree Insertions

1. Start with a **valid** red black tree.
2. Perform **naive BST insertion** algorithm, ignoring colors.
3. Set the new node to be **red**.

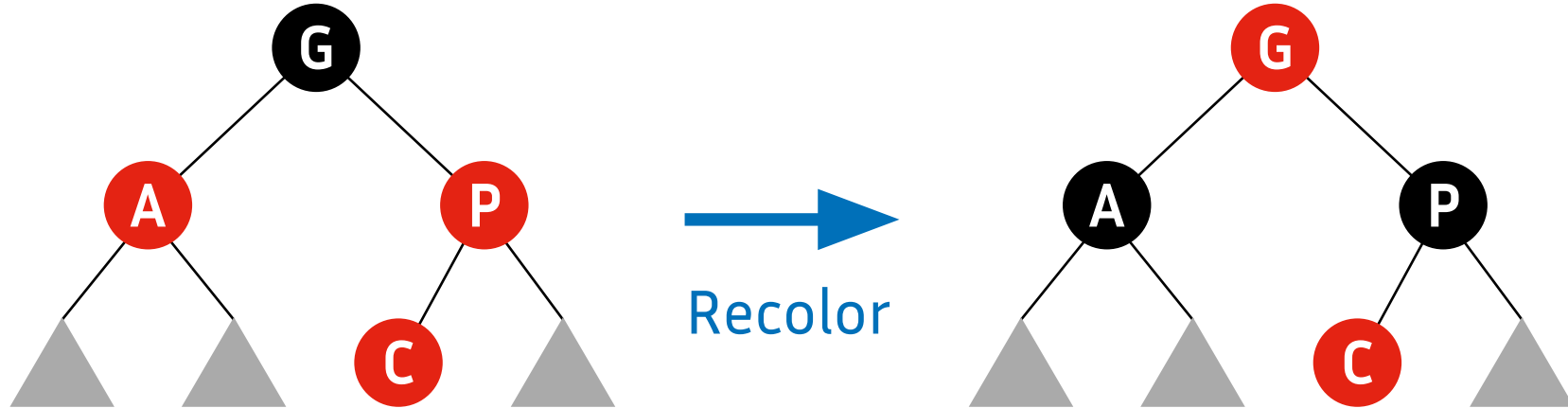
Red Black Tree Insertions

1. Start with a **valid** red black tree.
2. Perform **naive BST insertion** algorithm, ignoring colors.
3. Set the new node to be **red**.
4. **Fix** any new red black property violation.

Case 1: Red Aunt

If the aunt of the child is **red**:

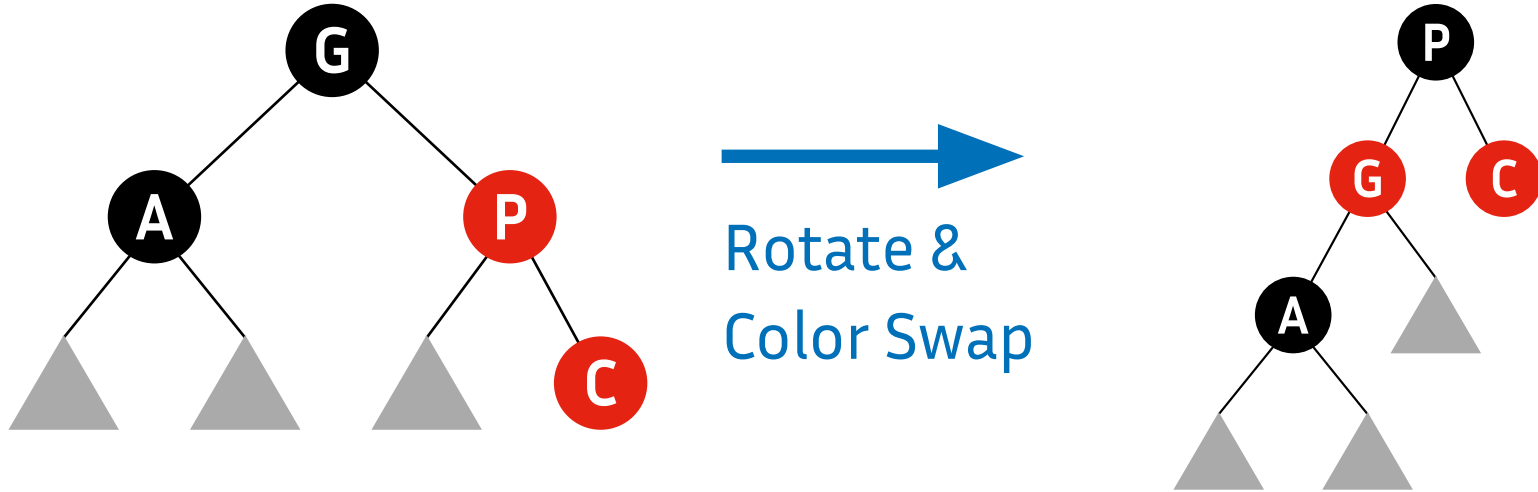
1. Make the parent and aunt **black**
2. Make the grandparent **red**
3. Check for red-child violations **higher up the tree**.



Case 2: Black Aunt Same Side

If the aunt of the child is **black** and the parent and child are on the **same side** (i.e. both left or both right children):

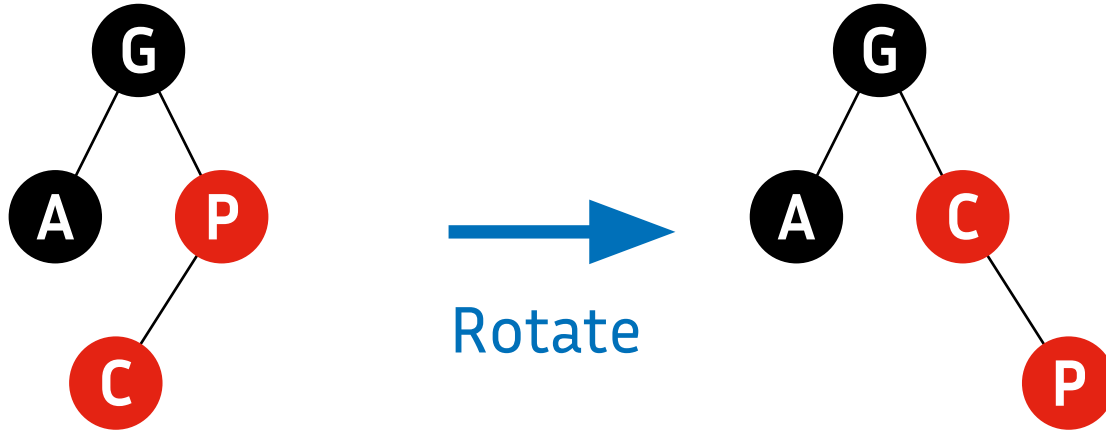
1. **Rotate** the parent and grandparent
2. Swap the colors of the parent and grandparent



Case 3: Black Aunt Zig-Zag

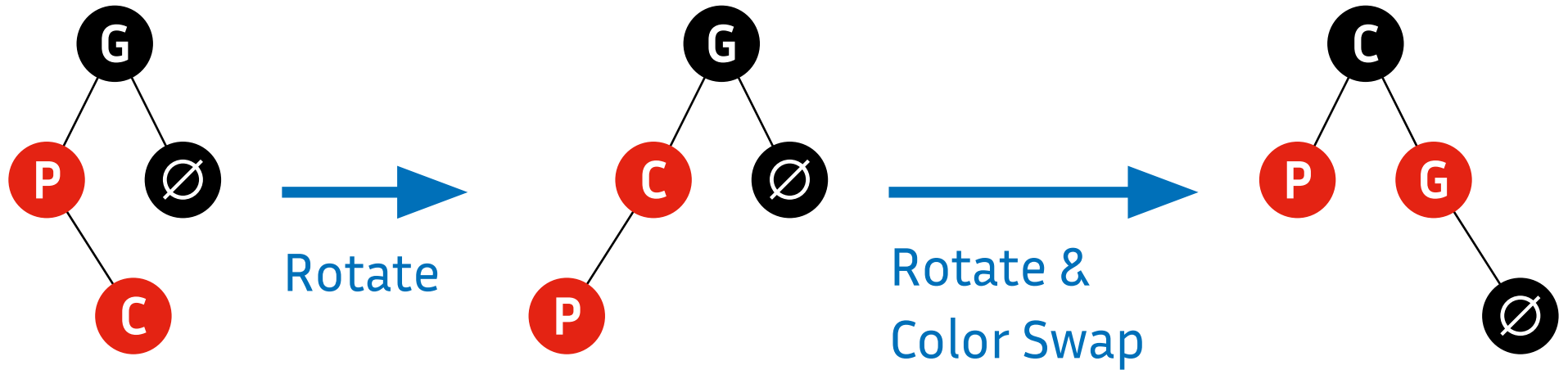
If the aunt of the child is **black** and parent and child are **on different sides** (not both left or both right children):

1. **Rotate** the parent and child
2. Handle the result using **Case 2**

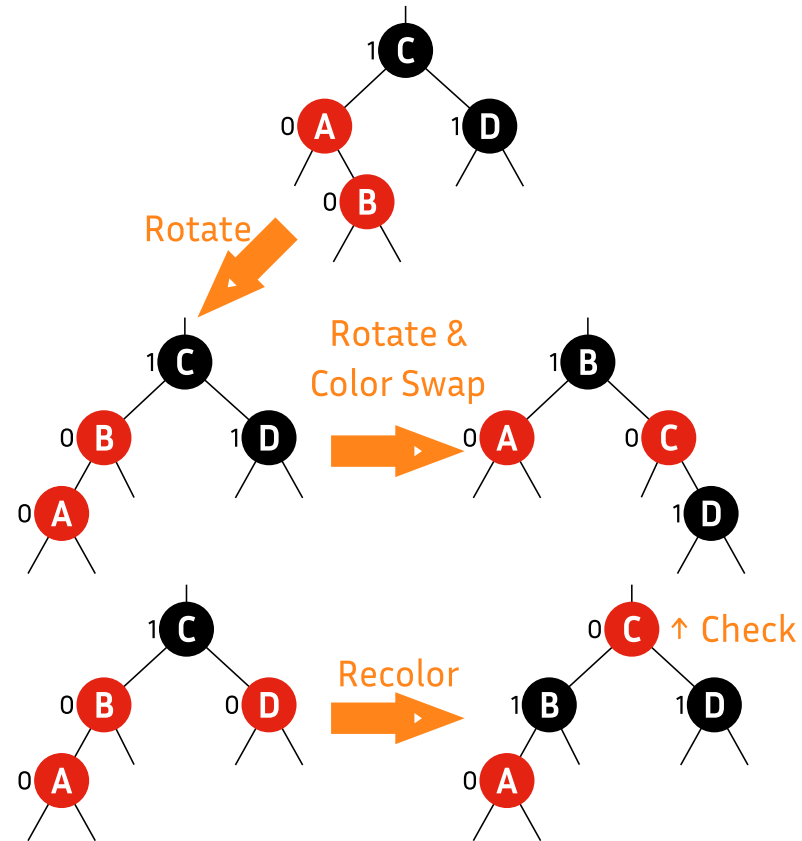


Case 4: Null Aunt

Use either **Case 2** or **Case 3** treating the null as if it was **black**.



Red Black Tree Insertion Summary

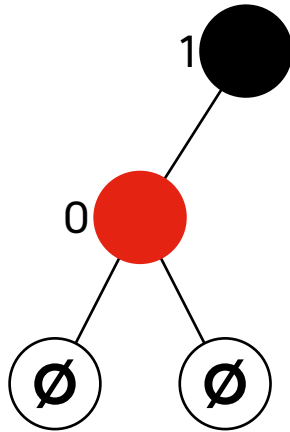


Red Black Tree Deletions

1. Start with a **valid** red black tree.
2. Perform **naïve BST deletion** algorithm, ignoring colors.
3. **Fix** any new red black tree property violation.

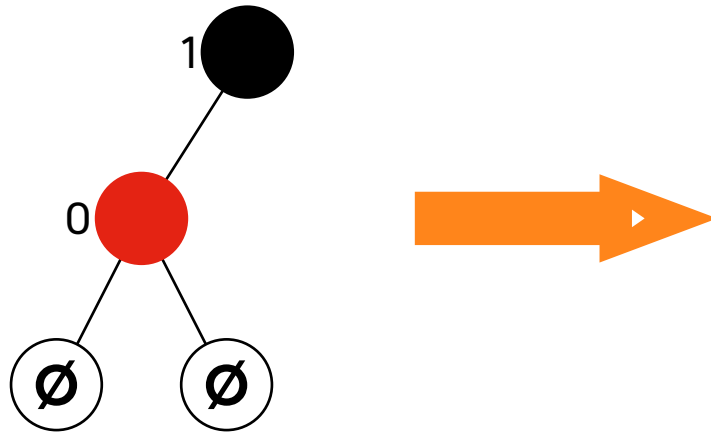
Remove Case: Red Leaf

If the node being deleted is a **red** leaf node, no extra steps are required.



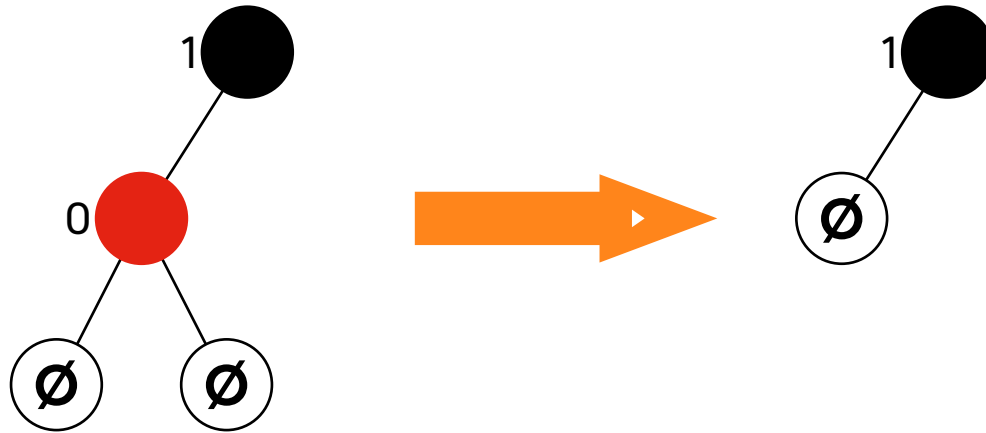
Remove Case: Red Leaf

If the node being deleted is a **red** leaf node, no extra steps are required.



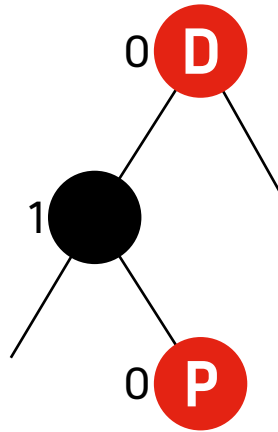
Remove Case: Red Leaf

If the node being deleted is a **red** leaf node, no extra steps are required.



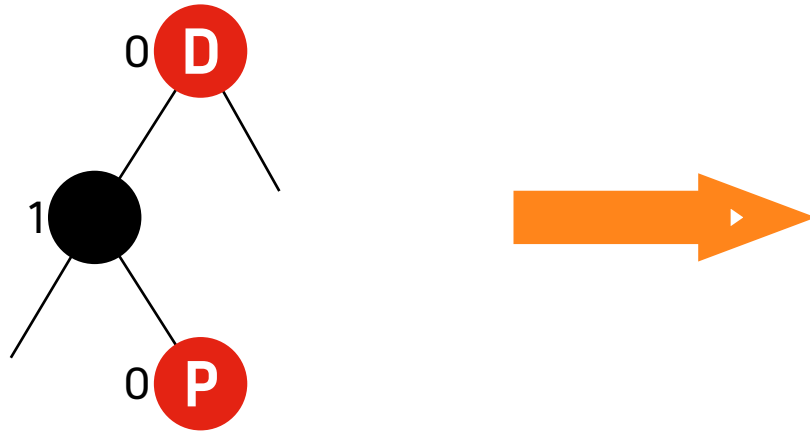
Remove Case: Red with Red Predecessor

If the node being deleted is **red**, and its predecessor is **red**, no extra steps are required.



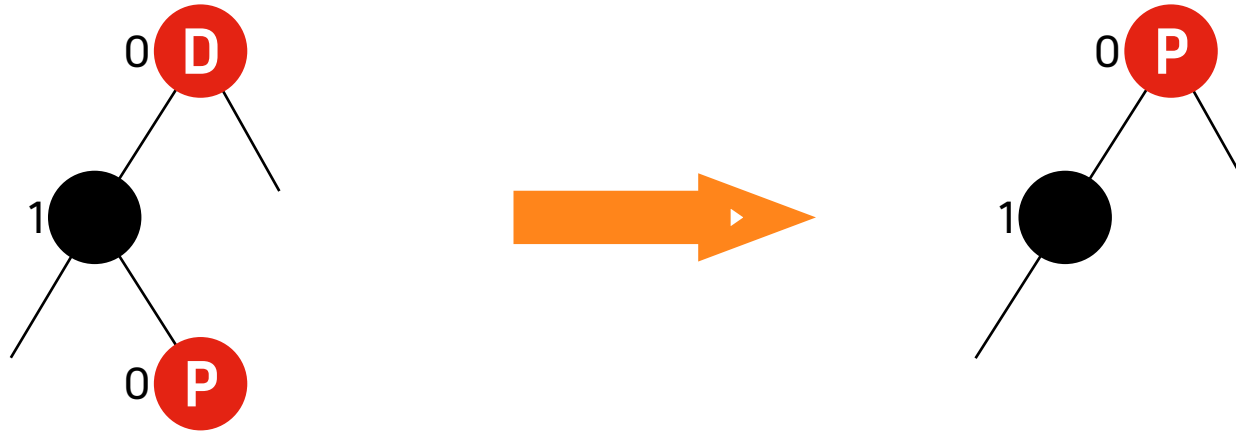
Remove Case: Red with Red Predecessor

If the node being deleted is **red**, and its predecessor is **red**, no extra steps are required.



Remove Case: Red with Red Predecessor

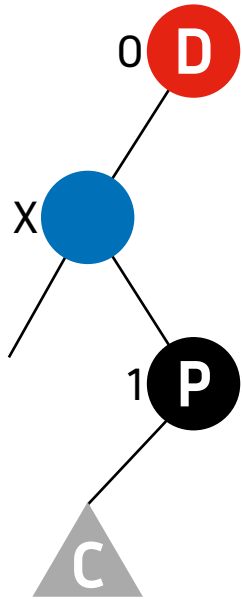
If the node being deleted is **red**, and its predecessor is **red**, no extra steps are required.



Remove Case: Red with Black Predecessor

If the node being deleted is **red**, and its predecessor is **black**:

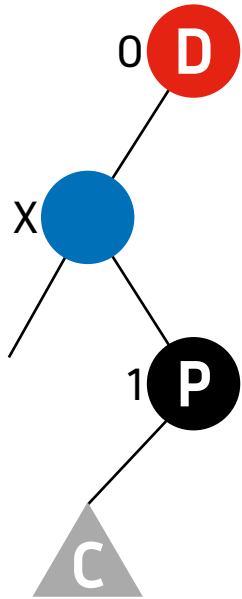
1. Add **+1** to the color of the predecessor's previous child.



Remove Case: Red with Black Predecessor

If the node being deleted is **red**, and its predecessor is **black**:

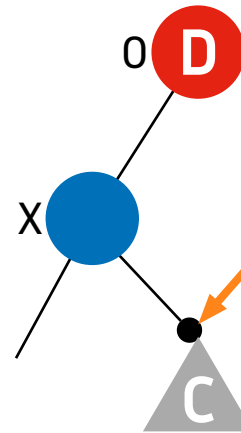
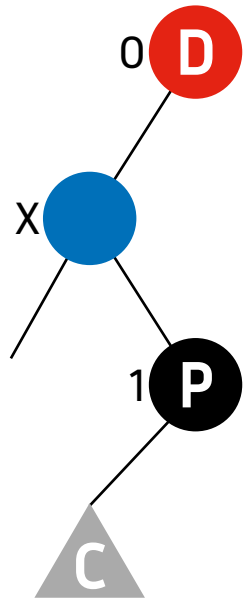
1. Add **+1** to the color of the predecessor's previous child.



Remove Case: Red with Black Predecessor

If the node being deleted is **red**, and its predecessor is **black**:

1. Add **+1** to the color of the predecessor's previous child.

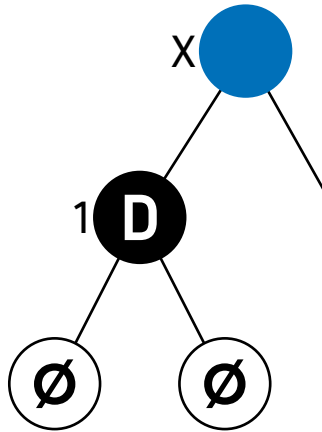


Need to add another black node along this path

Remove Case: Black Leaf

If the node being deleted is a **black** leaf node:

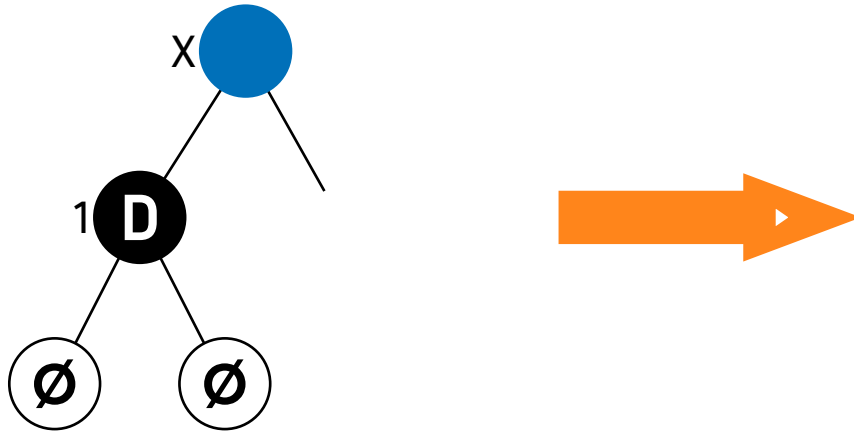
1. Add **+1** to the color of the null replacing the value.



Remove Case: Black Leaf

If the node being deleted is a **black** leaf node:

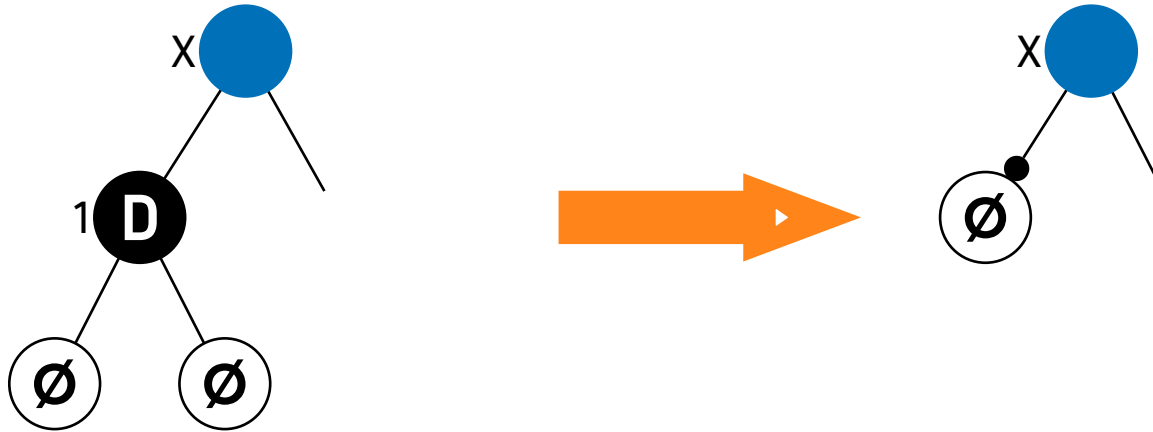
1. Add **+1** to the color of the null replacing the value.



Remove Case: Black Leaf

If the node being deleted is a **black** leaf node:

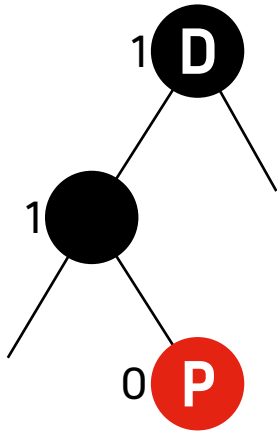
1. Add **+1** to the color of the null replacing the value.



Remove Case: Black with Red Replacement

If the node being deleted is **black**, and its replacement (either child or predecessor) is **red**:

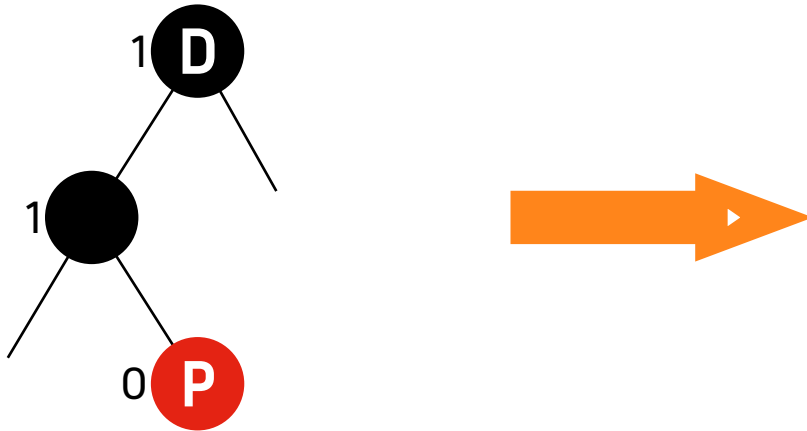
1. Set the replacement to **black**



Remove Case: Black with Red Replacement

If the node being deleted is **black**, and its replacement (either child or predecessor) is **red**:

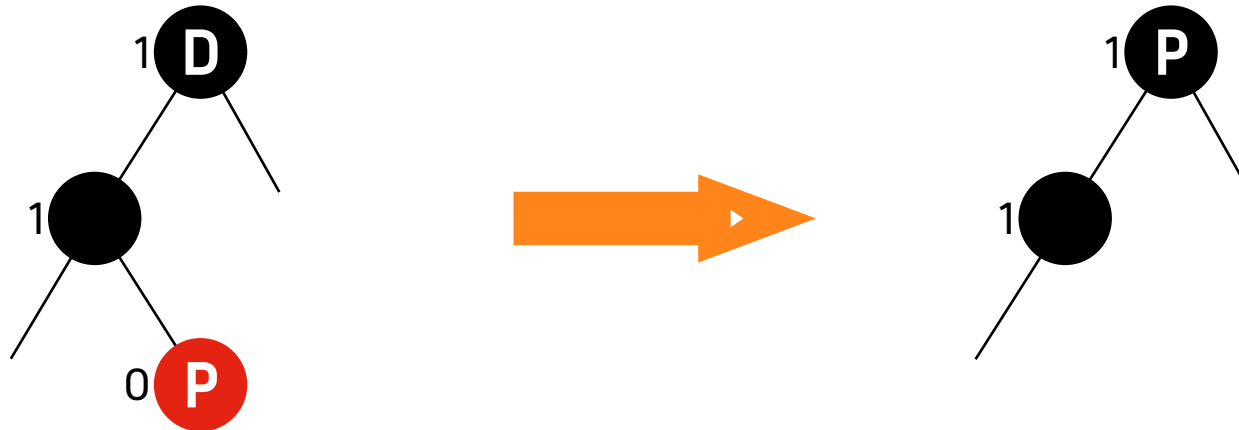
1. Set the replacement to **black**



Remove Case: Black with Red Replacement

If the node being deleted is **black**, and its replacement (either child or predecessor) is **red**:

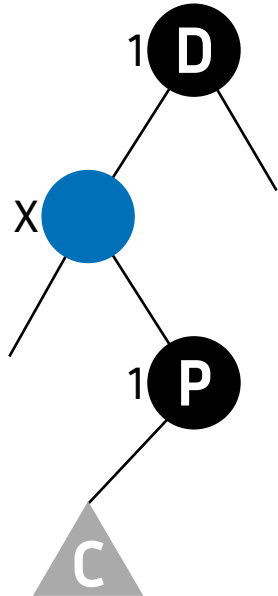
1. Set the replacement to **black**



Remove Case: Black with Black Replacement

If the node being deleted is **black**, and its replacement is **black**:

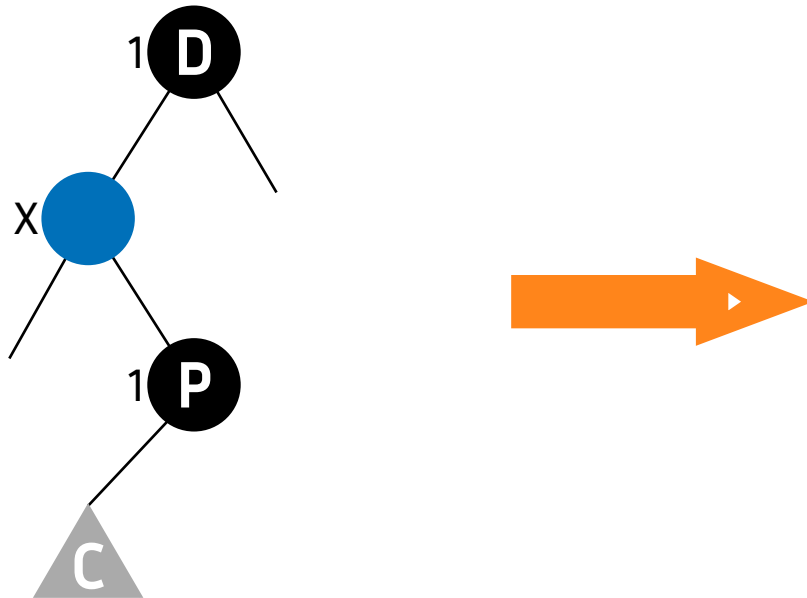
1. Add **+1** to the color of the predecessor's previous child



Remove Case: Black with Black Replacement

If the node being deleted is **black**, and its replacement is **black**:

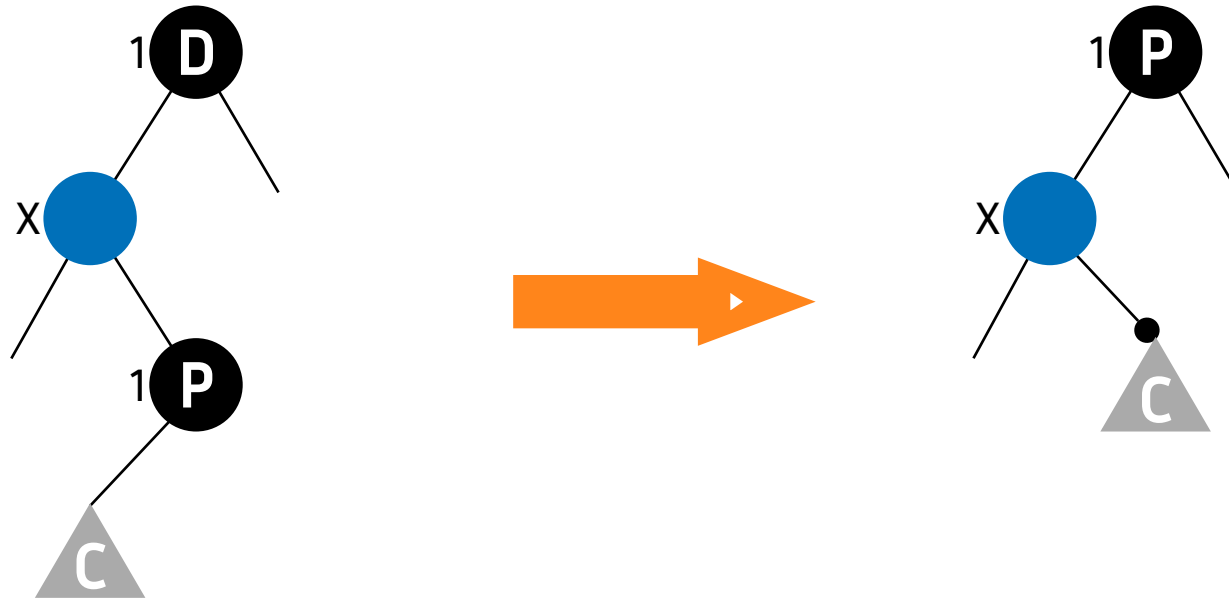
1. Add **+1** to the color of the predecessor's previous child



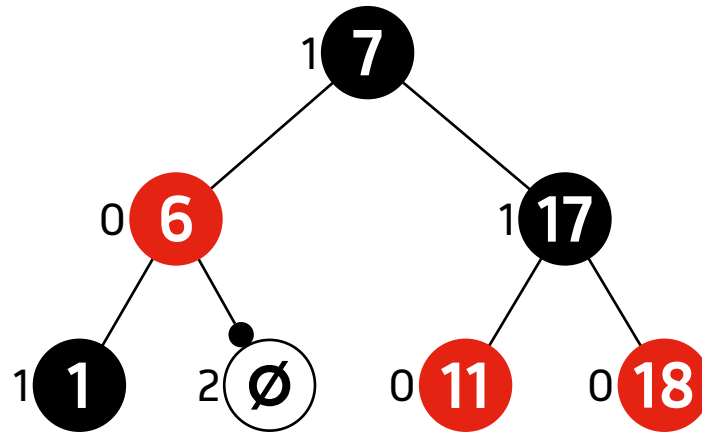
Remove Case: Black with Black Replacement

If the node being deleted is **black**, and its replacement is **black**:

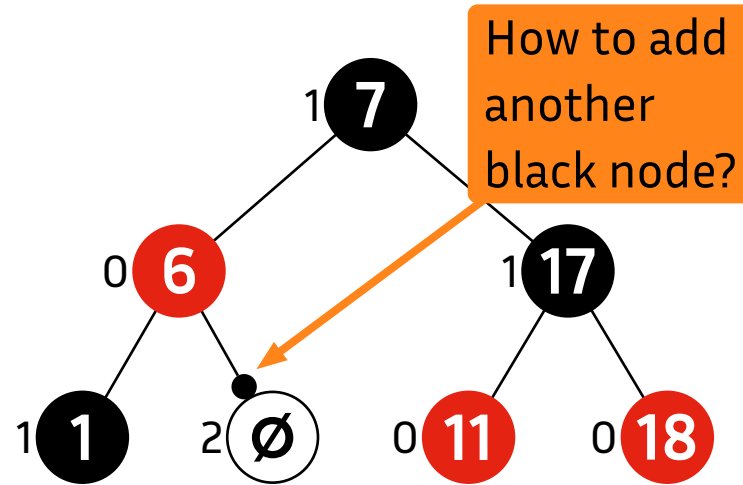
1. Add **+1** to the color of the predecessor's previous child



Black-Height Violations



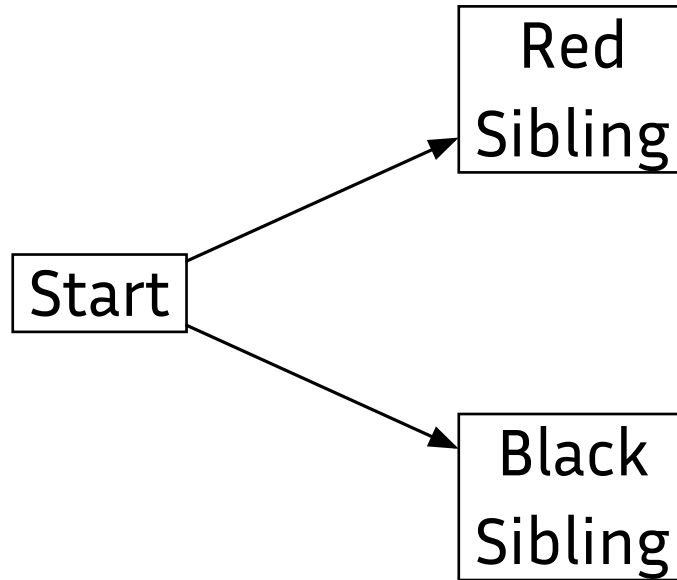
Black-Height Violations



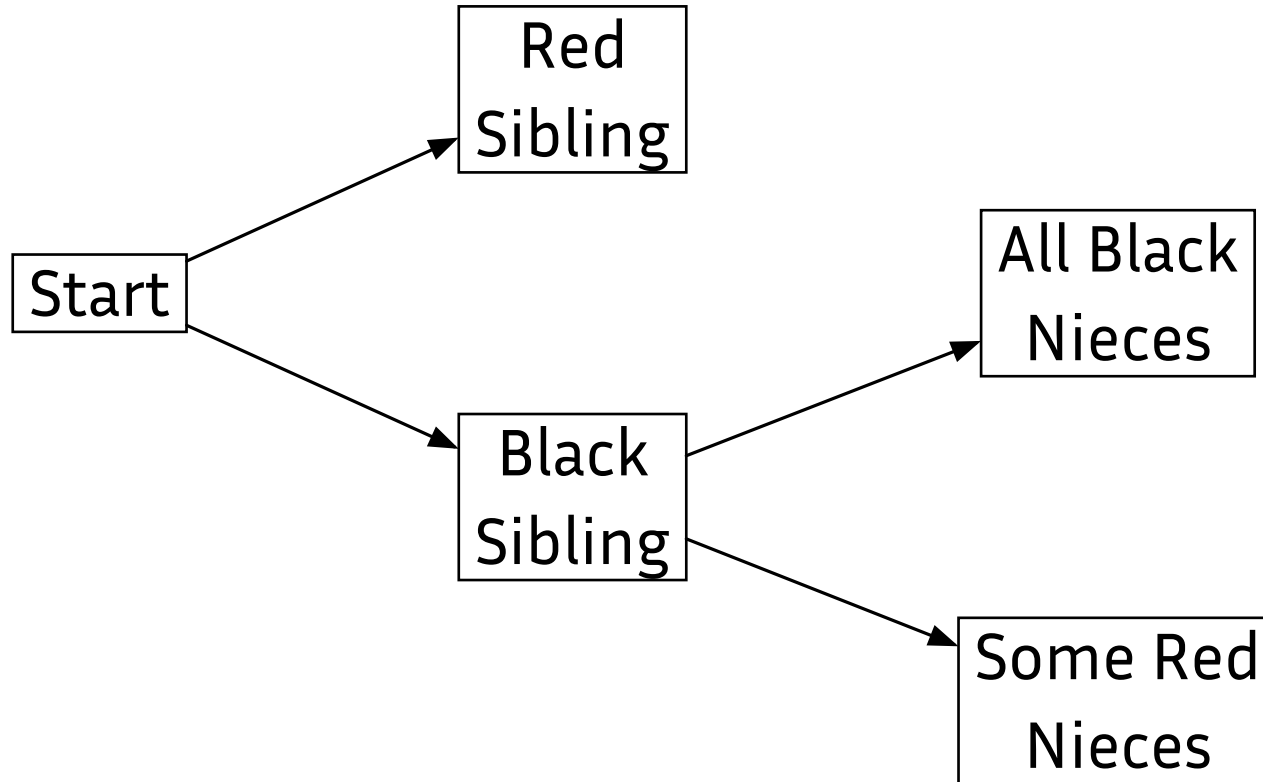
Marked Node Cases

Start

Marked Node Cases



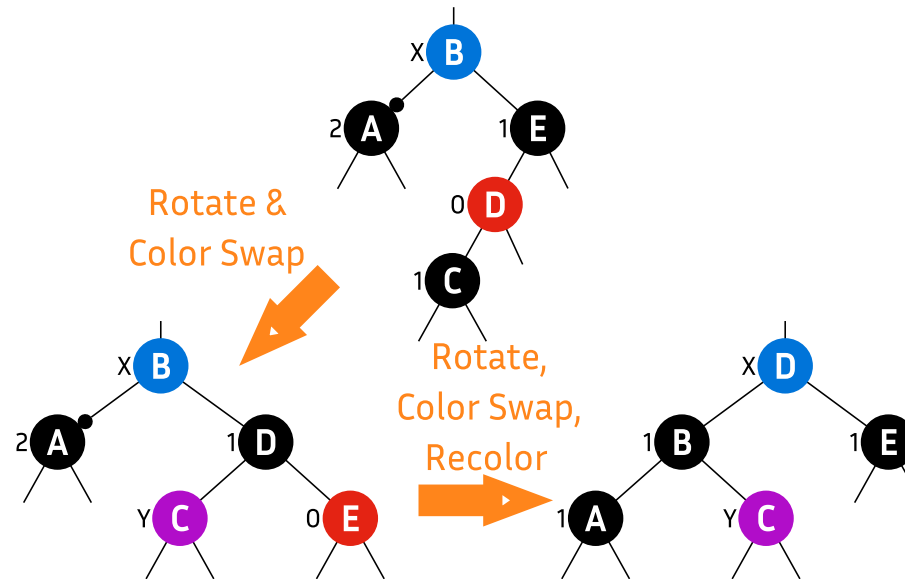
Marked Node Cases



Case 1: Black Sibling, Red Niece

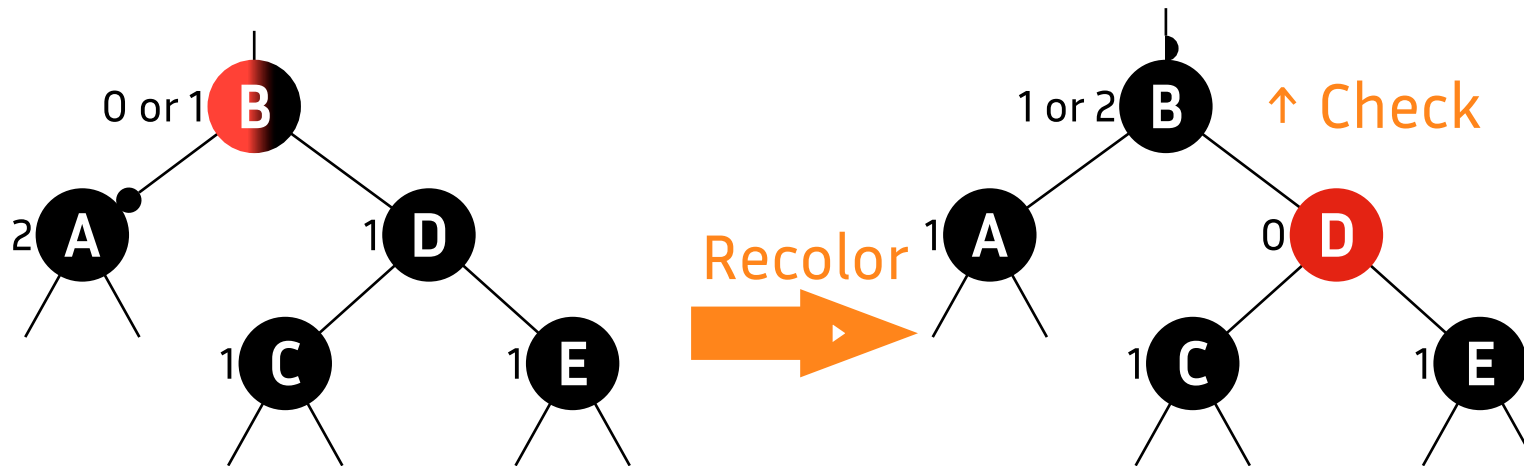
If the sibling of the marked node is **black** and has a **red** child:

- ✦ Only perform the first step if B, D and C are zig-zag.



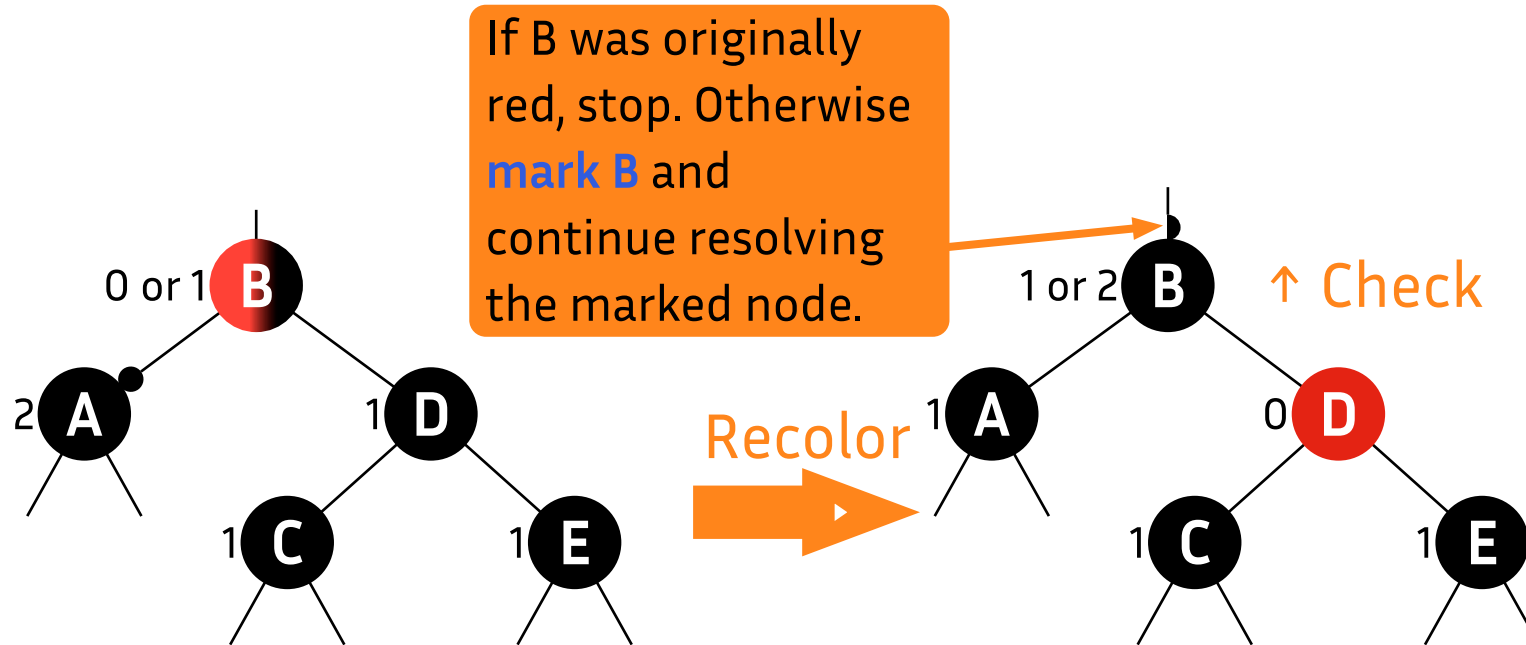
Case 2: Black Sibling, Black Niece

If the sibling of the marked node is **black**, and the sibling's children are all **black** (or null):



Case 2: Black Sibling, Black Niece

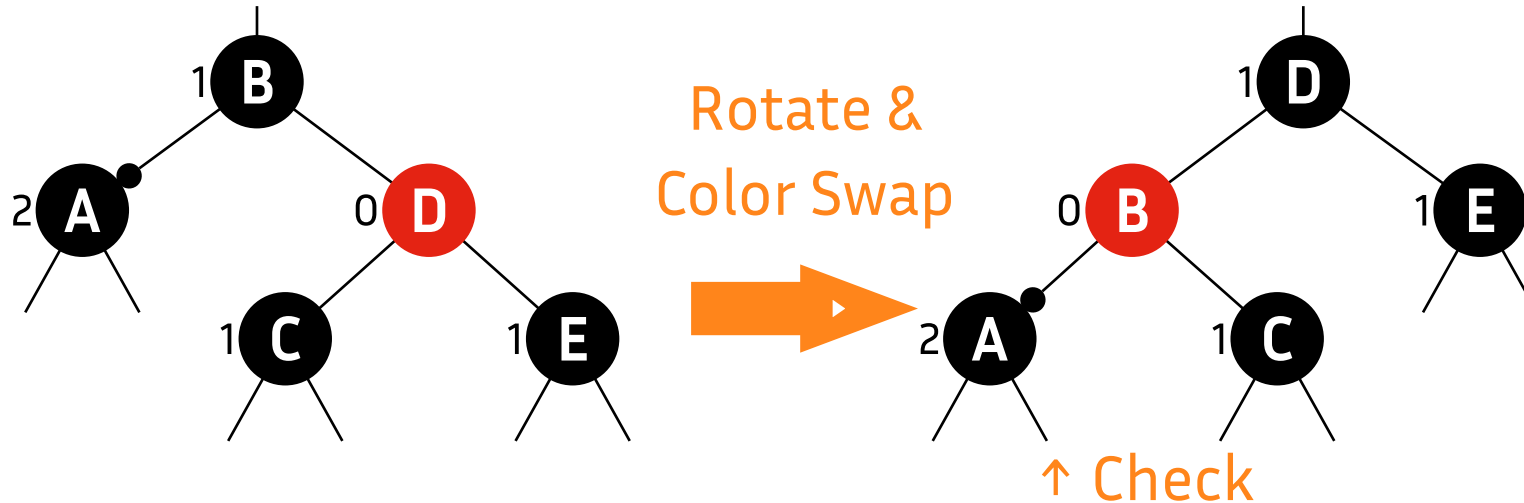
If the sibling of the marked node is **black**, and the sibling's children are all **black** (or null):



Case 3: Red Sibling

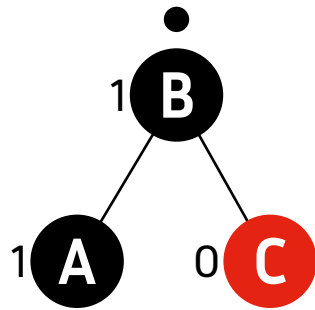
If the sibling of the marked node is **red**:

1. Rotate & color swap the parent and sibling
2. Continue to resolve the marked node



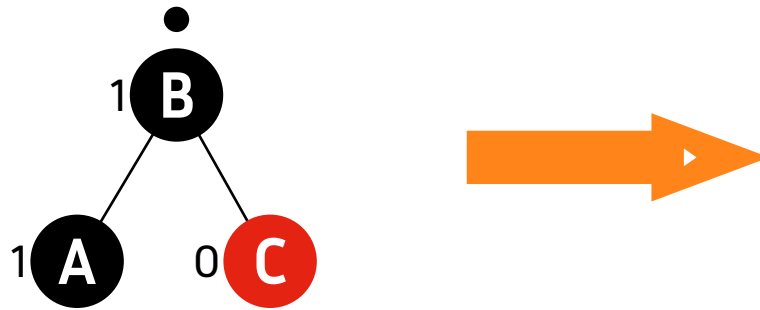
Case 4: Root

If the marked node is the root, remove the mark.



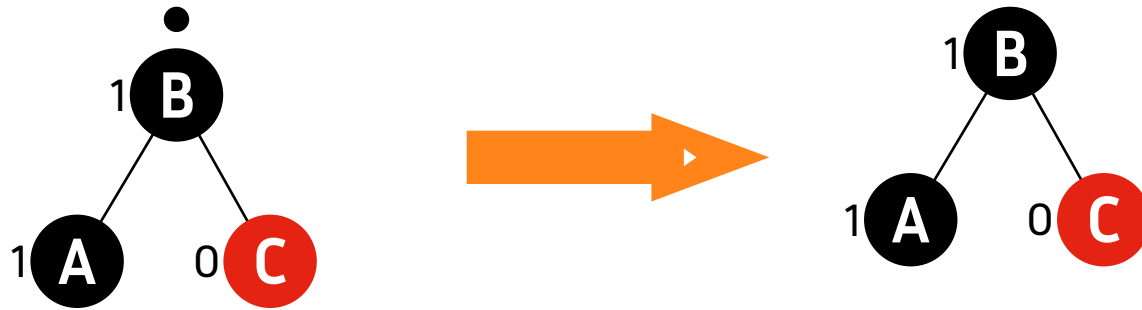
Case 4: Root

If the marked node is the root, remove the mark.

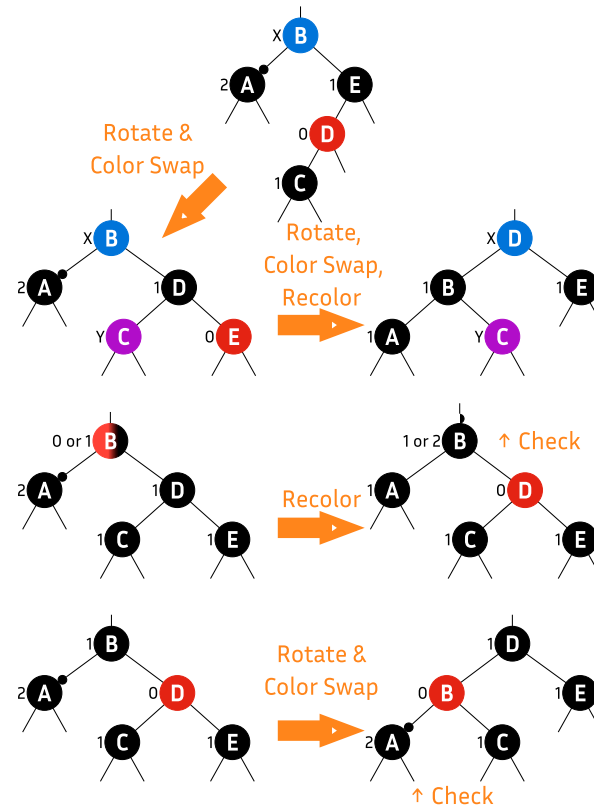


Case 4: Root

If the marked node is the root, remove the mark.



Marked Node Cases



AVL Trees

- ✦ **Adelson-Velsky and Landis (AVL) Trees** are self-balancing Binary Search Trees which use rotations to maintain balance.

AVL Trees

- ✦ **Adelson-Velsky and Landis (AVL) Trees** are self-balancing Binary Search Trees which use rotations to maintain balance.
- ✦ After each insertion / deletion, perform some **rotations** to **maintain the balance** of the tree.

AVL Trees

- ✦ **Adelson-Velsky and Landis (AVL) Trees** are self-balancing Binary Search Trees which use rotations to maintain balance.
- ✦ After each insertion / deletion, perform some **rotations** to **maintain the balance** of the tree.
- ✦ Rather than colors, track an explicit **balance factor**.

Balance Factor

Definition (Balance Factor) At a node v , the **balance factor** is the height of the left subtree minus the height of the right subtree.

$$\text{bf}(v) = \text{height}(\Delta_\ell^v) - \text{height}(\Delta_r^v)$$

- ✦ Where $\text{height}(\Delta_\ell^v)$ and $\text{height}(\Delta_r^v)$ give the height of the left and right subtrees of v respectively.

Balance Factor

Definition (Balanced Node) A node v is **balanced** if $|\text{bf}(v)| < 2$.

Balance Factor

Definition (Balanced Node) A node v is **balanced** if $|\text{bf}(v)| < 2$.

- ✦ $\text{bf}(v) < -1$ indicates that Δ_ℓ^v is smaller than Δ_r^v .
- ✦ $\text{bf}(v) > 1$ indicates that Δ_ℓ^v is taller than Δ_r^v .

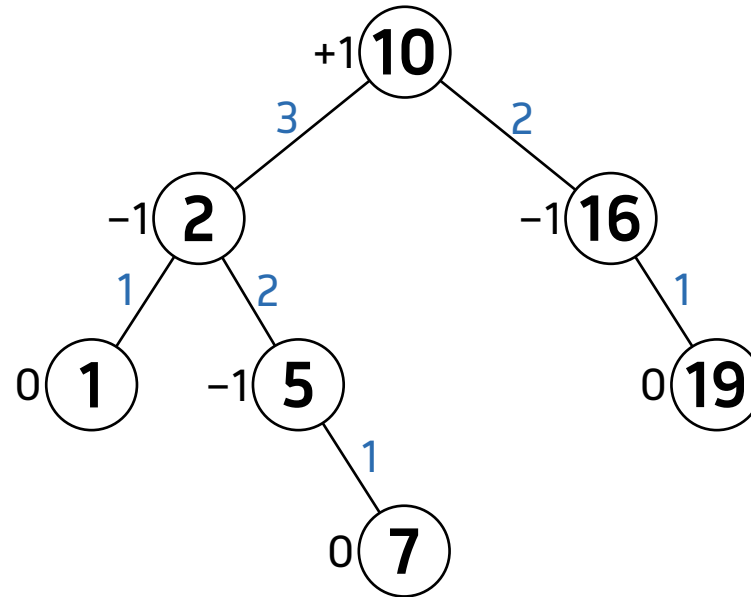
Balance Factor

Definition (Balanced Node) A node v is **balanced** if $|\text{bf}(v)| < 2$.

- ✦ $\text{bf}(v) < -1$ indicates that Δ_ℓ^v is smaller than Δ_r^v .
- ✦ $\text{bf}(v) > 1$ indicates that Δ_ℓ^v is taller than Δ_r^v .

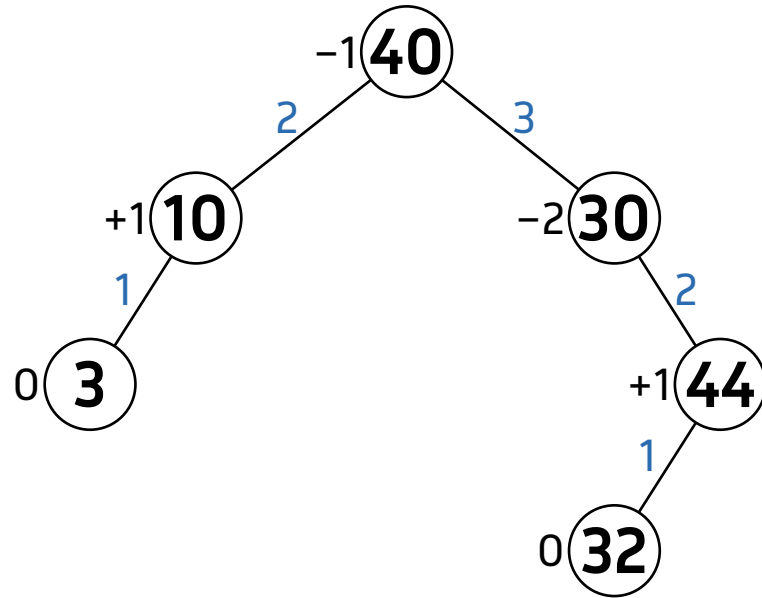
Definition (AVL Tree) An AVL Tree is a binary search tree where every node is **balanced**.

AVL Example

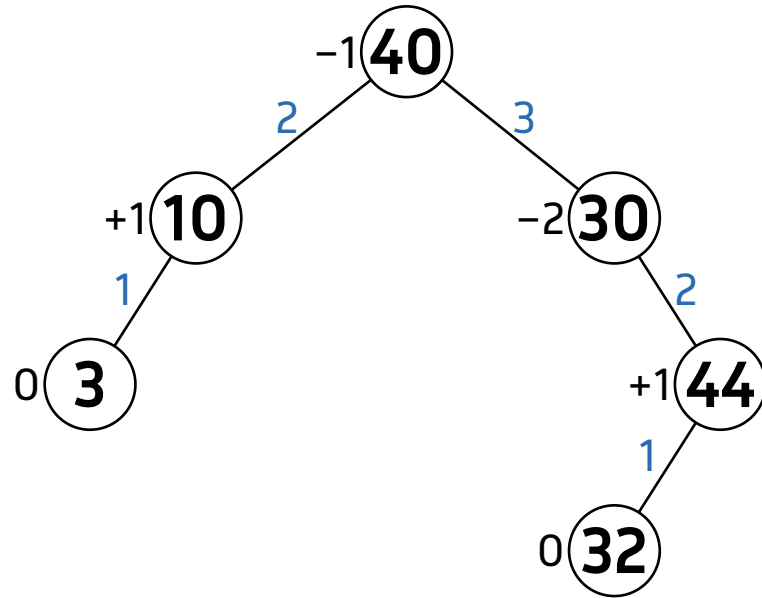


- ✦ Balance factor next to node
- ✦ Edges labeled with height of subtree rooted at the child.

AVL Example



AVL Example



- ✦ Balance factor of node 30 is -2 , so the tree is unbalanced!

Insertion Operation

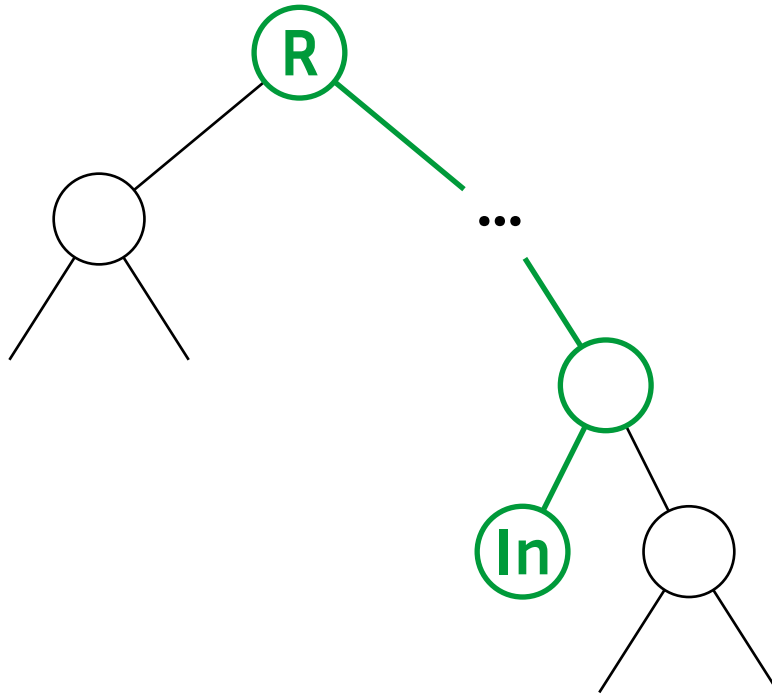
1. Start with a **valid AVL tree**.
2. Perform a **BST insertion**.
3. **Update** the balance factors from the inserted node to the root.
 - ✦ For each unbalanced node from the inserted one til the root:
 - ◇ **Rotate** to repair balance factor.
 - ◇ **Update** balance factors of rotated nodes.

Insertion Operation

- ✦ **Update** the balance factors from the inserted node to the root.

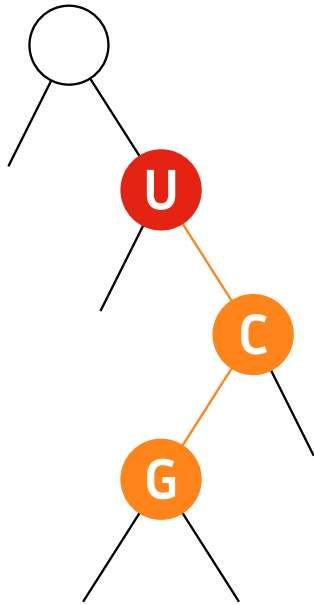
Insertion Operation

- ✦ **Update** the balance factors from the inserted node to the root.



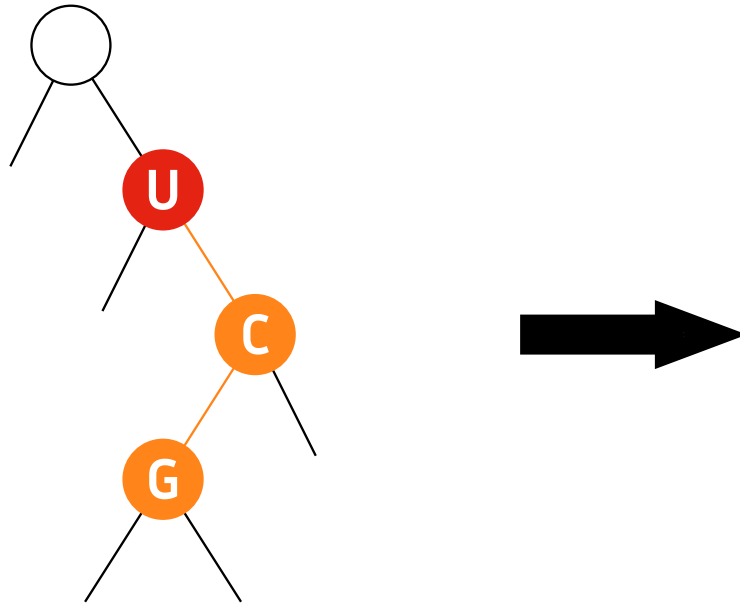
Rebalance

- ✦ If the grandchild is an *inner child*, **rotate** the grandchild and child on the heavy side.



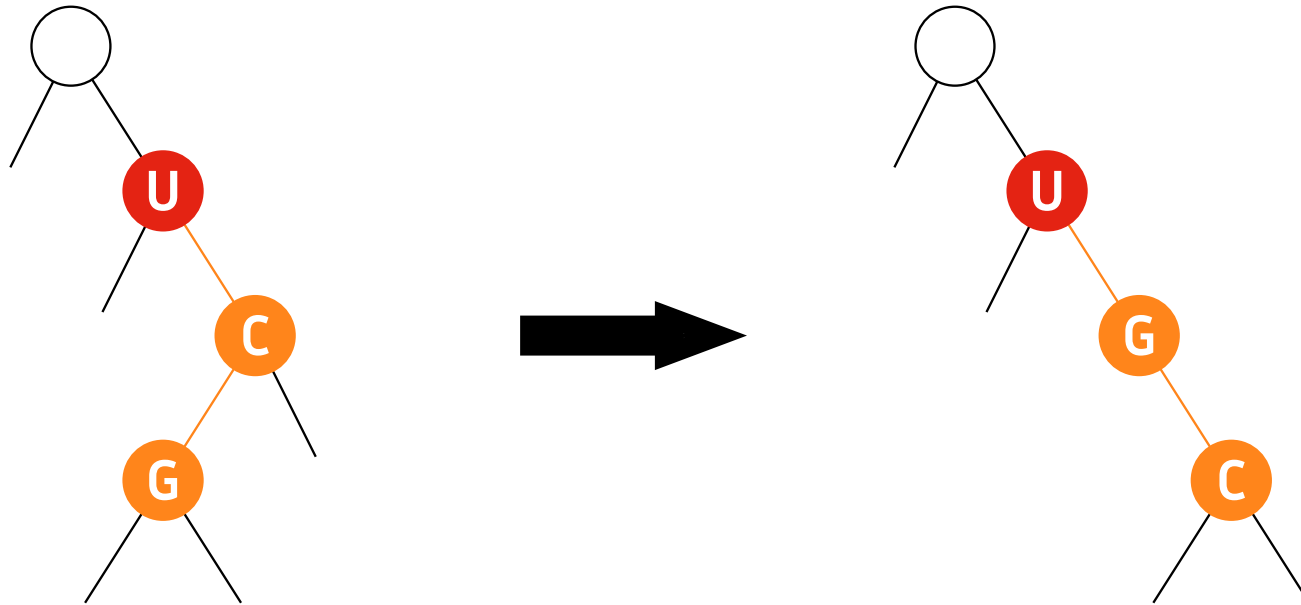
Rebalance

- ✦ If the grandchild is an *inner child*, **rotate** the grandchild and child on the heavy side.



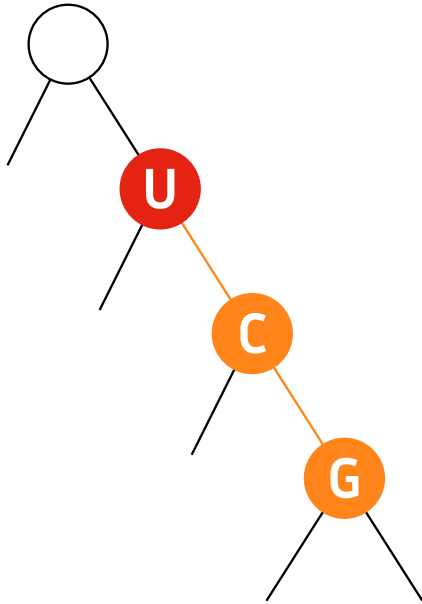
Rebalance

- ✦ If the grandchild is an *inner child*, **rotate** the grandchild and child on the heavy side.



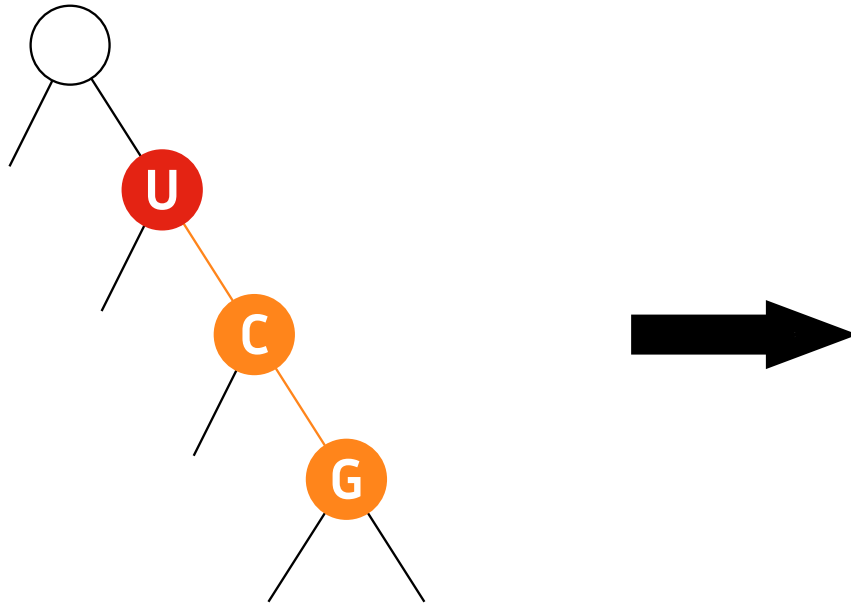
Rebalance

- ✦ **Rotate** the unbalanced node and its child node on the heavy side.



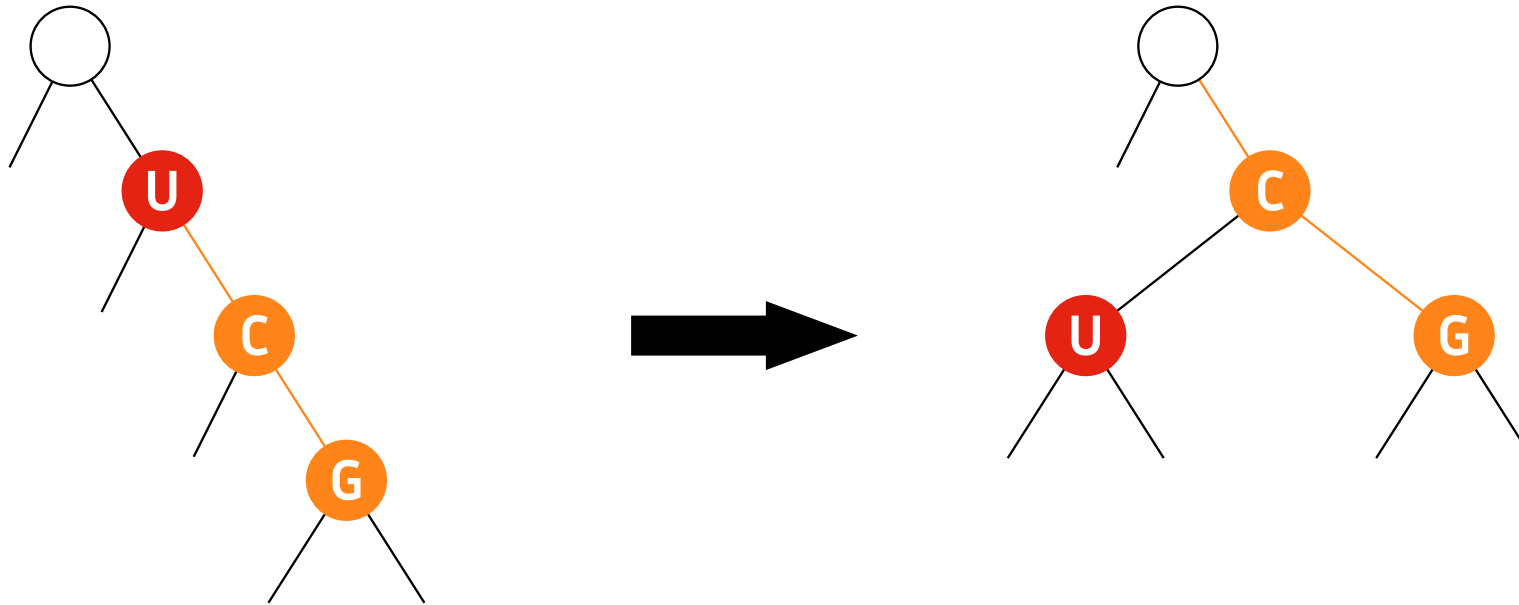
Rebalance

- ✦ **Rotate** the unbalanced node and its child node on the heavy side.



Rebalance

- ✦ **Rotate** the unbalanced node and its child node on the heavy side.

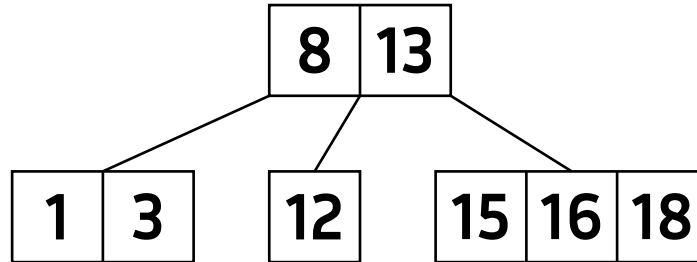


B-Tree Definition

- ✦ **B-Trees** are self-balancing trees where each node may have multiple children and / or multiple values.
- ✦ B-Trees are not binary search trees, but do follow an order property.

B-Tree Definition

- ✦ **B-Trees** are self-balancing trees where each node may have multiple children and / or multiple values.
- ✦ B-Trees are not binary search trees, but do follow an order property.



B-Tree Properties

- ✦ An internal node with n values has $n + 1$ children.

B-Tree Properties

- ✦ An internal node with n values has $n + 1$ children.
- ✦ All leaf nodes must be at the same level / depth.

B-Tree Properties

- ✦ An internal node with **n values** has **n + 1 children**.
- ✦ All **leaf** nodes must be at the same level / depth.
- ✦ **Ordering Properties**

B-Tree Properties

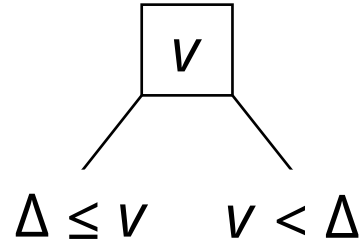
- ✦ An internal node with **n values** has **n + 1 children**.
- ✦ All **leaf** nodes must be at the same level / depth.
- ✦ **Ordering Properties**
 - ◇ The values in **each node** must be **sorted**.

B-Tree Properties

- ✦ An internal node with **n values** has **n + 1 children**.
- ✦ All **leaf** nodes must be at the same level / depth.
- ✦ **Ordering Properties**
 - ◇ The values in **each node** must be **sorted**.
 - ◇ All values in **subtrees** left of a value X must be smaller than X, and all values in **subtrees** right of X must be larger than X.

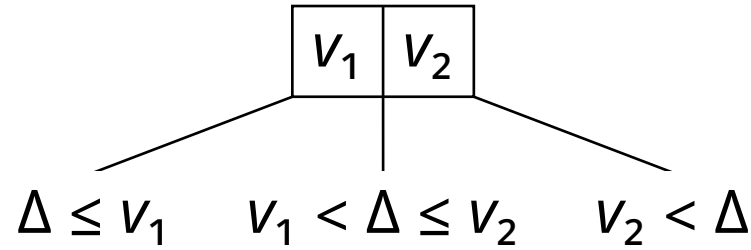
Internal Node Types

2 - Node



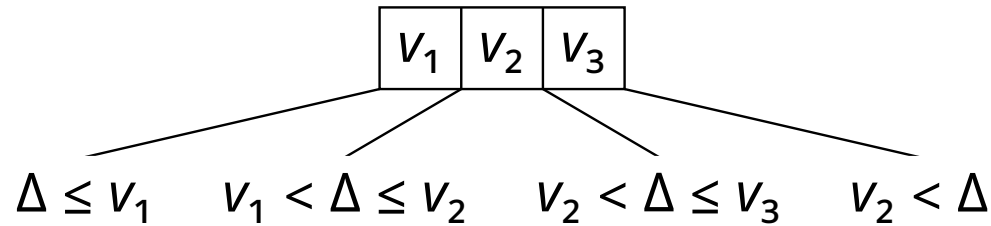
1 value, 2 children

3 - Node



2 values, 3 children

4 - Node



3 values, 4 children

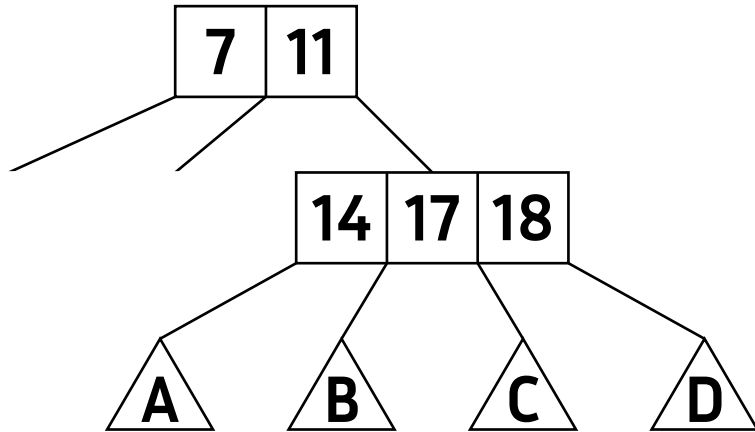
B-Tree Insertion

- ✦ Insertion in a B-Tree is always done in a leaf node.
- ✦ When a new level is needed, the tree grows at the root level.
- ✦ **Preemptive splitting** is when we split full nodes along the search path during insertion.

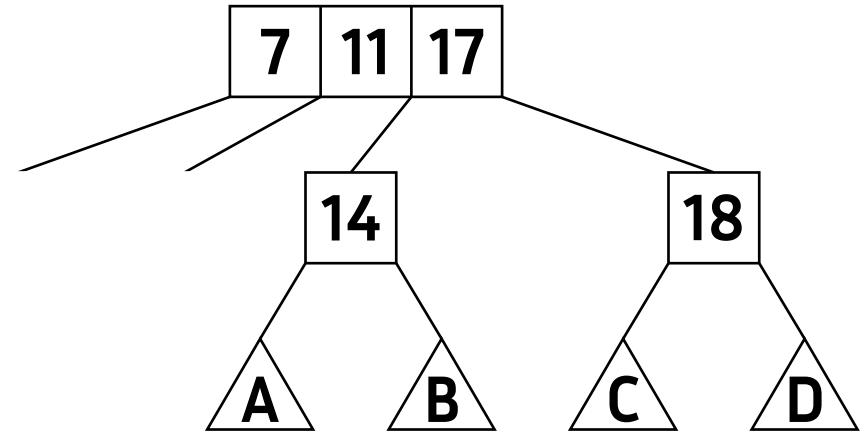
B-Tree Insertion Algorithm

1. Find where the value should go using the search algorithm.
 - a. For each “full” node along the way (including leaves), **preemptively split** the node, and continue the search at the new parent.
 - b. Don’t split the new parent, even if it’s full.
2. **Insert** new value **into leaf** at the end.

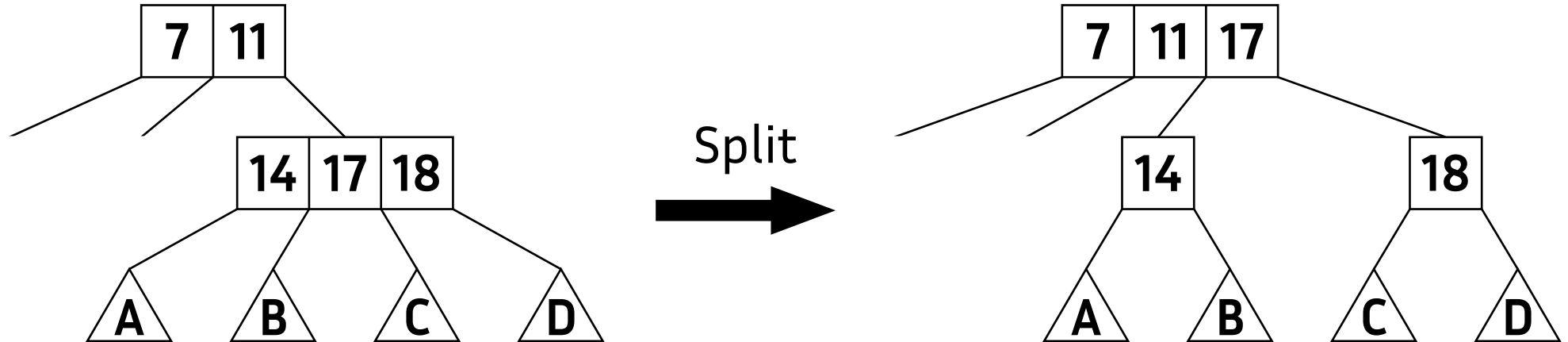
Preemptive Splitting



Split
➔



Preemptive Splitting



- ✦ Continue insertion search from parent node.



Practice Exam